

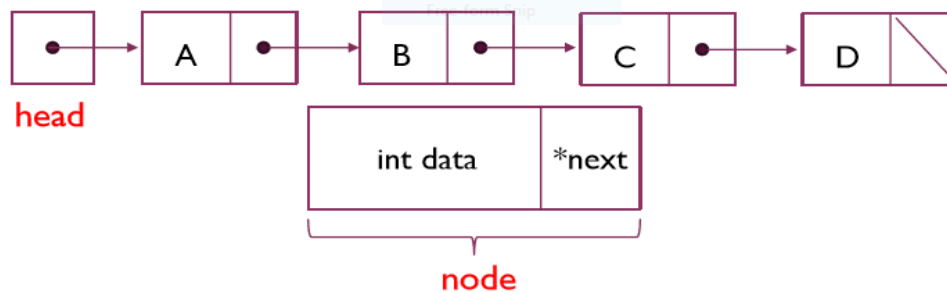
MODUL 3 (LINKED LIST I)

DESKRIPSI TEMA

Memory allocation adalah proses dimana program komputer memesan tempat di memory virtual.

Linked list adalah sebuah struktur data yang digunakan untuk menyimpan banyak objek data yang disimpan secara linear dan dinamis, sehingga memungkinkan kita untuk melakukan penambahan, pengurangan dan pencarian terhadap elemen data yang ada di linked list secara efektif.

Single linked list adalah bentuk sebuah linked list dimana sebuah node hanya sekali menunjuk ke node selanjutnya, sehingga pointer yang digunakan hanya satu arah. Representasi sederhana dari linked list dapat dilihat pada gambar di bawah.



Tahapan dalam pembuatan linked list:

1. Deklarasi tipe data abstrak dalam sebuah simpul (node).
2. Deklarasi pointer penunjuk kepala (head).
3. Membentuk simpul pertama dan mengaitkannya ke penunjuk kepala (head).
4. Membentuk simpul baru dan mengaitkannya ke linked list.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa dapat memahami dan menerapkan linked list (C₃).

PENUNJANG PRAKTIKUM

1. Software CodeBlocks

LANGKAH-LANGKAH PRAKTIKUM

Tutorial 1 – Memory Allocation

1. Mengalokasikan suatu variabel pointer

```
#include<stdio.h>

int main(){
    int *arr;
    arr=(int*) malloc(sizeof(int*));
}
```

2. Mengalokasikan sejumlah N array of pointer

```
int *arr,i;
arr = (int*) malloc(sizeof(int*)*100);
```

3. Mengalokasikan array of pointer, lalu untuk setiap array of pointer yang sudah dialokasikan. Alokasikan kembali setiap pointer sejumlah N elemen

```
#include<stdio.h>

int main(){
    int **arr,i;
    arr=(int**) malloc(sizeof(int*)*100);
    for(i=0;i<100;i++){
        arr[i]=(int *) malloc(sizeof(int)*100);
    }
}
```

4. Melakukan pembebasan memory seperti cara melakukan pengalokasiannya

```
for(i=0;i<100;i++){
    free(arr[i]);
}
free(arr);
```

Tutorial 1.1

1. Buatlah program yang terdapat proses alokasi memory untuk array 3 dimensi dengan nama file **Week[xx]_[NIM]_[nama]_01.c**.
Contoh : **Week04_00000013728_Mario_01.c**.

Tutorial 2 – Declaration

Berdasarkan penjelasan pada tutorial sebelumnya, Anda sudah mengetahui konsep dari single linked list, pada tahapan ini Anda akan mempelajari bagaimana cara mendeklarasikan single linked list. Adapun tahapannya adalah:

1. Deklarasi tipe data abstrak

Contoh dalam melakukan deklarasi adalah:

```
struct nama_struct {  
    <tipe_data> <nama_variabel>;  
    ...  
    struct nama_struct <nama_variabel_pointer>;  
}
```

Contoh dalam bentuk kode:

```
struct contoh{  
    int data;  
  
    struct contoh *next;  
}
```

2. Deklarasi penunjuk kepala (head)

Penunjuk / pointer kepala digunakan untuk menandakan node (simpul) pertama dalam suatu linked list.

Contoh deklarasi pointer head:

```
int main (int argc, const char *argv[]) {  
    struct nama_struct <nama_variabel_pointer_kepala>;  
    ...  
    <nama_variabel_pointer_kepala> = NULL;  
    return 0;  
}
```

Contoh dalam bentuk kode:

```
int main(int argc, char const *argv[]){  
    struct contoh *head;  
  
    head = NULL;  
  
    return 0;  
}
```

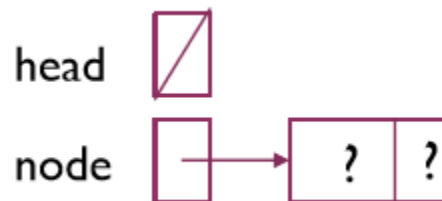
3. Membentuk simpul pertama dan mengaitkannya ke penunjuk kepala (head)

Untuk membentuk suatu simpul pertama, mula-mula kita harus menyiapkan tempat dalam memori.

Caranya adalah sebagai berikut:

- a. Memesan sebuah tempat di memori untuk simpul baru :

```
node = (struct contoh*) malloc(sizeof(struct contoh));
```



- b. Masukkan data ke simpul

```
int bil;
scanf("%d", &bil);
node->data = bil;
```



Tutorial 3 – Add, Print, & Free

Tutorial 3.1 – Add Node Single Linked List

Konsep penambahan simpul ke dalam single linked list dapat dilakukan dengan beberapa macam cara. Salah satu cara untuk menambahkan simpul pada single linked list adalah dengan menyambungkan node di depan penanda head.

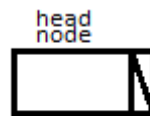
Simpul baru untuk data pertama dapat dituliskan dengan kode sebagai berikut:

```

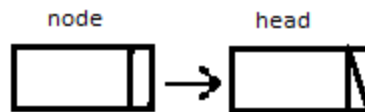
if (head == NULL) {
/*
    pada kondisi ini menyatakan bahwa linked list belum memiliki
    simpul satupun maka ketika memasukkan simpul pertama, kondisi
    ini akan dijalankan
*/
    head = node;    //1
}
node->next = head; //2
head = node;      //3

```

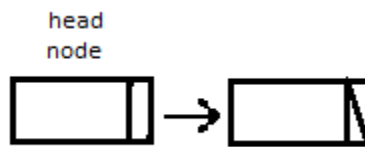
Representasi linked list dapat digambarkan sebagai berikut:



Gambar 1 Representasi gambar untuk //1



Gambar 2 Representasi gambar untuk //2



Gambar 3 Representasi gambar untuk //3

Tutorial 3.2 – Print

Data dalam linked list dapat dicetak isinya.

Langkah-langkah yang dibutuhkan untuk mencetak isi dari linked list adalah sebagai berikut:

1. Tambahkan sebuah pointer variabel *temp* (atau gunakan yang sudah ada dan sudah tidak digunakan)
2. Assign *temp* sebagai head
3. Cetak data yang ada di dalam *temp*

4. Lanjutkan hingga *temp*->next
5. Ulangi sampai dengan *temp*->next = NULL

Contoh sederhana dalam bentuk kode adalah sebagai berikut:

```
struct contoh *temp = head; //1 & 2
while (temp != NULL) { //5
    printf("%d\n", temp->data); //3
    temp = temp->next; //4
}
```

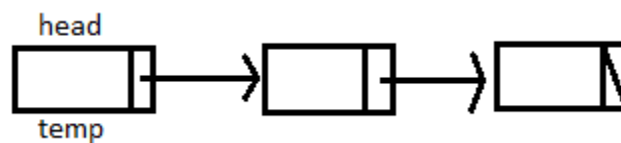
Tutorial 3.3 – Free

Setelah linked list selesai digunakan dalam sebuah program, memory harus dibersihkan agar tidak penuh.

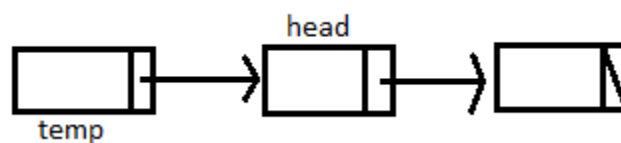
Penghapusan sebuah linked list tidak hanya kepala (head) saja, melainkan semua node yang terhubung harus dihapus.

Langkah-langkah sederhananya adalah sebagai berikut:

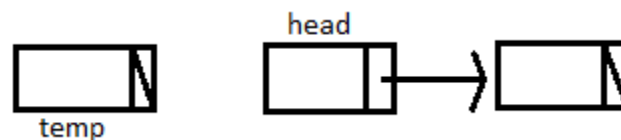
1. Siapkan sebuah variabel temporary yang nantinya akan digunakan sebagai penanda node yang akan dihapus
2. Set temporary sebagai head



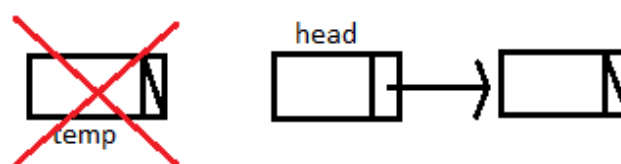
3. Letakkan head ke node selanjutnya



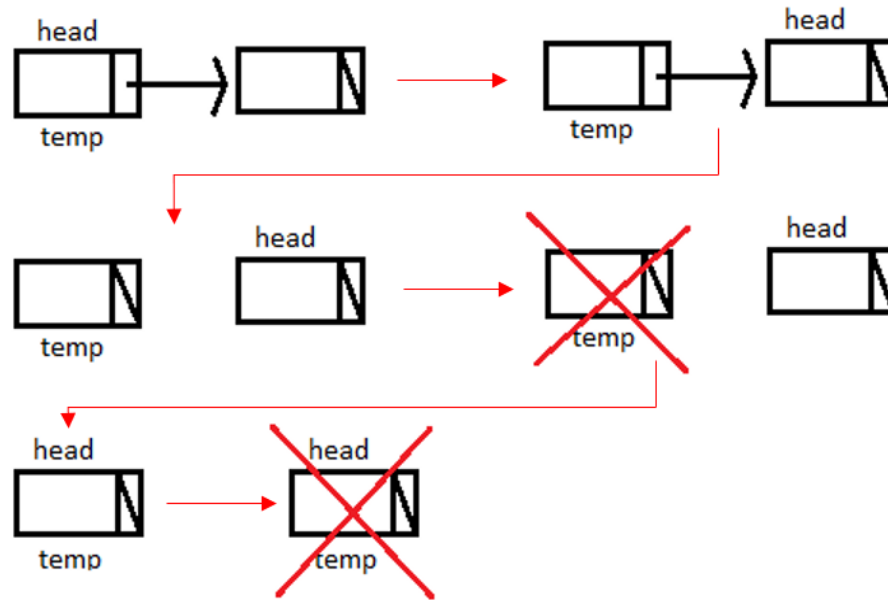
4. Set temp->next menjadi NULL



5. Hapus node temp dengan menggunakan free(temp)



6. Ulangi langkah 2 hingga node pada linked list habis



Tutorial 3.4 - Let's make it!

Setelah membaca semua kondisi dan pengertian yang ada di atas mengenai linked list, mari kita membuat kode tersebut.

1. Buatlah sebuah file dengan nama **Week[xx]_[nim]_[nama]_02.c**.
Contoh: **Week04_00000013728_Mario_02.c**
2. Ketiklah kode yang ada di bawah ini dan jelaskan maksud line yang diberikan command huruf.
Tuliskan penjelasan tersebut pada file yang sama !

```
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>
#include <string.h>
#include <conio.h>

typedef struct orang {
    char nama[30];
    int usia;

    struct orang *next;
} orang;

int main(int argc, char const *argv[]) {
    orang *head, *curr, *node; //<a> jelaskan kegunaan 3 variabel dalam potongan kode ini
    char nama[30];
    int usia, exit=1;
    char lanjut;

    head = curr = NULL;

    while(exit == 1) {
        system("cls");
        printf("Data diri yang akan dimasukkan dalam linked list\n");
        printf("-----\n\n");
        printf("Masukkan nama: "); scanf("%[^\n]", &nama);
        printf("Masukkan usia: "); scanf("%d", &usia);
        getchar();

        node = (struct orang*) malloc(sizeof(struct orang)); //<b> apa kegunaan baris kode berikut?
        node->next = NULL;

        strcpy(node->nama, nama);
        node->usia = usia;
```



```

if(head == NULL) { //<c> apa kegunaan baris kode berikut?
    head = node;
}
else { //<d> apa kegunaan baris kode berikut? apa yang membedakannya dengan <c>?
    curr = head;
    while(curr->next != NULL) {
        curr = curr->next;
    }
    curr->next = node;
}

//<e> apa tujuan dari looping ini (pilih)?
//a. kode yang tidak ada gunanya || b. error handling sederhana || c. pemanjang barisan kode
do {
    printf("Lanjut (y/n)? "); scanf("%c", &lanjut);
    getchar();
    if(lanjut == 'y' || lanjut == 'Y') {
        exit = 1;
        break;
    }
    else if(lanjut == 'n' || lanjut == 'N') {
        exit = 0;
        break;
    }
    else {
        printf("Masukkan input yang benar !\n");
        printf("Tekan tombol apapun untuk melanjutkan. . .\n");
        getch();
    }
} while(1);
}

system("cls");
printf("Daftar nama yang terdapat dalam linked list\n");
printf("-----\n\n");

curr = head;

```

```

while(curr != NULL) { //<f> apa yang dilakukan dalam pengulangan berikut?
    printf("Nama : %s\nUsia : %d\n", curr->nama, curr->usia);
    printf("-----\n");
    curr = curr->next;
}

while(head != NULL) {
    //<g> Tuliskan perintah untuk melakukan free memory dari head
    // gunakan contoh potongan kode pada Free sebagai petunjuk
}

return 0;
}

```

Challenge 4

Challenge 4.1

Ubahlah array pada challenge week 3 (Challenge 3.1 dan Challenge 3.2) dengan memory allocation.

Challenge 4.2

Buatlah sebuah program yang memiliki ketentuan sebagai berikut:

Contoh Menu:

```
=====
Menu peminjaman ruangan
=====
1. Pinjam ruangan
2. List Ruangan yang dipinjam
3. Hapus list ruangan yang dipinjam(Clear All)
4. Exit
Input :
```

Input merupakan integer.

Jika Menu Pinjam ruangan dipilih user harus memasukan datanya :

```
=====
Input Data Peminjaman ruangan
=====
Input nama      : Justin Susanto
Input NIM       : 0000013613
Program Studi   : Informatika
Angkatan        : 2016
Gedung (A-D)    : B
Nomor Ruangan   : 501
Data peminjaman ruangan telah berhasil diinput dengan status Unapproved !
Press any key to continue . . .
```

Jika Menu 2 dipilih maka akan menampilkan data yang telah diinput oleh user :

```

-----
Data ke-1
Nama      : Justin Susanto
Nim       : 0000013613
Prodi     : Informatika
Angkatan  : 2016
Ruangan   : B501
Status    : Unapproved
-----
Data ke-2
Nama      : James Christian
Nim       : 00000013678
Prodi     : Informatika
Angkatan  : 2016
Ruangan   : B503
Status    : Unapproved
-----
Data ke-3
Nama      : Mario Alexander
Nim       : 00000013841
Prodi     : Informatika
Angkatan  : 2016
Ruangan   : A720
Status    : Unapproved
-----
Press any key to continue . . .

```

Jika Menu Clear All dipilih :

```

Clear All Success !
Press any key to continue . . .

```

Interface menu tidak harus sama, tolong terapkan konsep linked list yang telah dipelajari pada tutorial tutorial sebelumnya.

Bonus :

```

=====
Input Data Peminjaman ruangan
=====
Input nama      : Mario Alexander
Input NIM       : 00000013728
Program Studi   : Informatika
Angkatan        : 2016
Gedung (A-D)    : B
Nomor Ruangan   : 503
Ruangan telah dipinjam !
Press any key to continue . . .

```

Bobot Penilaian :

1. Penggunaan struktur data linked list pada opsi pertama (termasuk memasukan data ke linkedlist dan membuat list!) [Score : 60]
2. Output data pada linked list, sesuai dengan yang diinginkan atau diinput [Score : 25]
3. Hapus Semua linked list yang telah dibuat (Clear All) [Score : 15]

